



Arduino XY2-100 and FB4 Signal Generator User Manual and Documentation

September 3, 2025

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Key Features	2
2	System Requirements	2
2.1	Hardware Requirements	2
2.2	Software Requirements	2
3	Installation Guide	2
3.1	Arduino IDE Installation	2
3.2	Uploading the Firmware	2
4	Circuit Connections	2
4.1	Arduino Portenta C33 Pinout (EXAMPLE)	2
4.2	Power Considerations	3
5	Protocol Configuration	3
5.1	Pin configuration	3
5.2	Waveform Selection	4
5.3	Frequency and Amplitude Configuration	4
5.4	Sample Rate Configuration	4
6	Protocol Implementation Details	4
6.1	XY2-100 Protocol	4
6.2	FB4 Protocol	5
6.3	Data Format Comparison	6
7	Operation Guide	6
7.1	Initial Setup	6
8	Appendices	7
8.1	Version History	7

1 Introduction

1.1 Purpose

The Arduino XY2-100 and FB4 Signal Generator is a template toolkit designed to generate precise signals compatible with both XY2-100 and FB4 protocols for controlling galvanometer scanners and laser projection systems. The application supports multiple data formats and provides independent waveform generation for X and Y channels with configurable parameters.

1.2 Key Features

- Protocol support: XY2-100 (16 and 18 bits) and FB4
- Multiple waveform types: sine, square, triangle, sawtooth
- Configurable frequency and amplitude for each channel, and adjustable sample rate

2 System Requirements

2.1 Hardware Requirements

- Arduino compatible board
- Pangolin Technology laser scanner
- Pangolin Mach-DSP and Power supply
- Mach-DSP software

2.2 Software Requirements

- Arduino IDE 1.8.x or later
- Required libraries: None (uses standard Arduino functions)

3 Installation Guide

3.1 Arduino IDE Installation

1. Download Arduino IDE from <https://www.arduino.cc/en/software>
2. Run the installer and follow the setup instructions
3. Verify installation by launching the Arduino IDE

3.2 Uploading the Firmware

1. Connect your Arduino board to the computer
2. Open the provided sketch (XY2-100 or FB4) in Arduino IDE
3. Select the correct board and port from the Tools menu
4. Click the Upload button to program the Arduino

4 Circuit Connections

4.1 Arduino Portenta C33 Pinout (EXAMPLE)

In theory, any Arduino can be configured to operate the XY2-100 or FB4 protocols for communication with the Mach-DSP. However, attention must be paid to the logic level voltage of the selected Arduino. The Mach-DSP operates at 3.3V logic, while many Arduinos use 5V; in these cases, a logic level shifter converter is required for proper communication. For this demonstration, an Arduino Portenta C33 was chosen because it operates at 3.3V. Additionally, the Portenta C33 provides multiple digital I/O pins suitable for signal generation. The recommended connections are shown in Figure 1.

Protocol	Portenta Pin	Signal	Description
XY2-100	D2	FS	Frame Sync
XY2-100	D9	SCLK	Serial Clock
XY2-100	D8	XDATA	X Channel Data
XY2-100	D7	YDATA	Y Channel Data
FB4	D2	FS	Frame Sync
FB4	D9	SCLK	Serial Clock
FB4	D8	XYDATA	Combined X/Y Data

Table 1: Portenta C33 pin connections

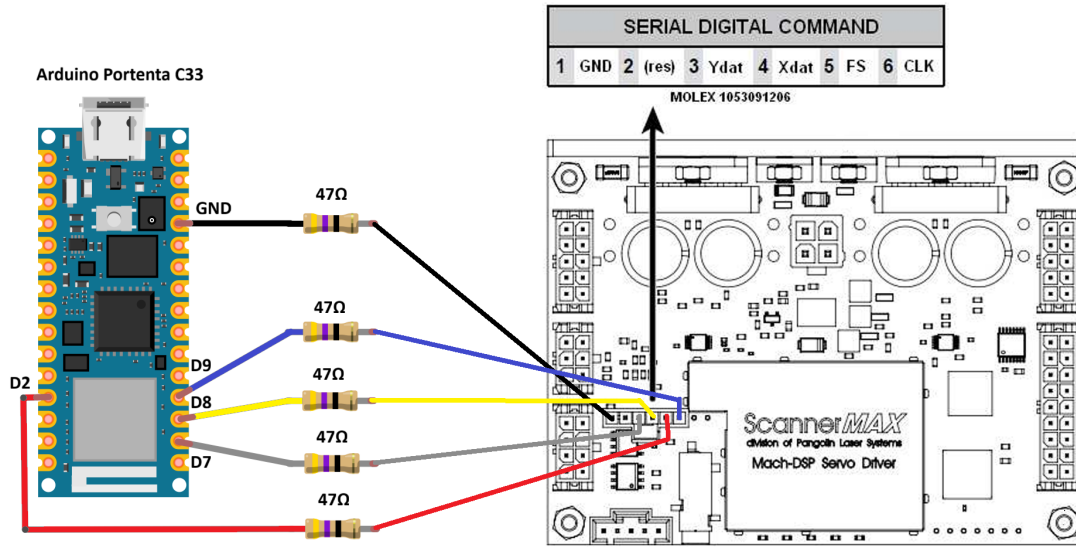


Figure 1: Connection diagram for Arduino Portenta C33 with XY2-100/FB4 interface

4.2 Power Considerations

- The Portenta C33 operates at 3.3V logic levels
- Most Arduino systems operate with 5V logic level - use level shifters converter if necessary
- Ensure proper grounding between Arduino and scanner system
- Signals are connected directly to the DSP processor, without protection of any kind. You should exercise care and use series-termination resistors. In the presented example 47-ohms resistors are used as shown in Figure 1.

5 Protocol Configuration

5.1 Pin configuration

The software developer kit supports two protocols. Each library XY2-100.ino or FB4.ino presented different pin configurations:

```

1 // XY2-100 Protocol Pins
2 const int FS_PIN = 2;    // Frame Sync
3 const int SCLK_PIN = 9;  // Serial Clock
4 const int XDATA_PIN = 8; // X Data
5 const int YDATA_PIN = 7; // Y Data

```

Listing 1: Pin Definitions

```

1 // FB4 Protocol Pins
2 const int FS_PIN = 2;    // Frame Sync
3 const int SCLK_PIN = 9;  // Serial Clock
4 const int XYDATA_PIN = 8; // Combined X and Y Data

```

Listing 2: Pin Definitions

5.2 Waveform Selection

The firmware supports multiple waveform types that can be independently configured for X and Y channels:

```
1 // CONFIGURATION - Set your desired waveforms here
2 const WaveformType X_WAVEFORM = WAVE_TRIANGLE;
3 const WaveformType Y_WAVEFORM = WAVE_SQUARE;
```

Listing 3: Waveform Configuration

Available waveform types:

- WAVE.SINE - Sine wave
- WAVE.SQUARE - Square wave
- WAVE.TRIANGLE - Triangle wave
- WAVE.RISING.SAW - Rising sawtooth wave
- WAVE.FALLING.SAW - Falling sawtooth wave

5.3 Frequency and Amplitude Configuration

```
1 // INDEPENDENT CHANNEL SETTINGS
2 #define DATA_AMPLITUDE_X 41071L // X channel amplitude
3 #define DATA_AMPLITUDE_Y 41071L // Y channel amplitude
4 #define FREQ_X 20 // X channel frequency (Hz)
5 #define FREQ_Y 20 // Y channel frequency (Hz)
```

Listing 4: Frequency and Amplitude Settings

5.4 Sample Rate Configuration

```
1 const int SAMPLE_RATE = 5000; // Samples per second (Adjustable)
```

Listing 5: Sample Rate Setting

6 Protocol Implementation Details

6.1 XY2-100 Protocol

The XY2-100 protocol is a serial protocol used for controlling galvanometer scanners. Looking at the diagram below, you will note that signals that are sent to the Mach-DSP must change on the low-to-high transition of the clock; signals are observed by the Mach-DSP on the high-to-low transitions of the clock.

Reception of a new Input Command begins when the Frame Sync line is brought high, followed by a high-to-low transition of the Clock line. During this same high-to-low clock transition, the first bit of data is also latched by the Mach-DSP. (Technically this is called "active-high, late frame sync" because the sync signal coincides with data reception.)

Once the Frame Sync line has been detected high, it can remain high during the next 18 clocks, or it may be brought low again. In any event, the Frame Sync line must be brought low by clock number 20, as shown in the diagram above, otherwise a Frame Sync Error will occur, and data will not be communicated correctly. In essence, all of this means that the data is "framed" in 20-clock sequences.

The Clock line can be any frequency up to a theoretical maximum of 25MHz, with any reasonable duty cycle. The data frames described above may occur in sequence, one after another (called "coherent frames"), or they may be sporadic (called "isolated frames"). If they are sporadic, the Clock line may continue clocking in between data frames, or it may be quiescent (preferably held in a low state). In all cases, whether the clock is operational or not, the Frame Sync line must be held low when data is not sent.

The "frame rate" may be anything that the Clock and Frame Sync signals allow - ranging from slower than one frame per hour up to a theoretical maximum of 1,250,000 frames per second. The data communicated via this serial port is absorbed by the Mach-DSP at the rate that it is sent, and - unless Synchronous Sampling is selected (as described several pages below), there is no attempt to synchronize the Mach-DSP's servo sample clock (typically 100kHz to 300kHz) to the serial port signals.

You will note that 19-bits of data per axis (and per returned status) are communicated using this protocol, although the entire sequence requires 20 clocks. And although parity is not used by this protocol, some data integrity is provided by "Control Bits".

You will also note that the first three bits received by the Mach-DSP are designated "C2, C1, C0". These first three bits are called "control bits", and they designate the function of the data that is contained in the bits that follow. The table below describes the control bits and associated functions:

Control Bits [2:0]	Function
000	(reserved for future use)
001	16-bit position data follows
010	(reserved for future use)
011	(reserved for future use)
1xx	18-bit position data follows

Table 2: Control Bit Definitions

As shown in the table, only Control Bit patterns 001, and 1xx are presently used, designating either 16-bit or 18-bit position commands, and therefore there is room for future expansion as well as flexibility to implement custom protocols for individual clients.

For 16-bit position data, a value of 0 (all bits cleared) normally represents the far-left and far-bottom of the scan field; a value of 32768 represents the middle of the scan field; and a value of 65535 (all bits set) normally represents the far-right and far-top of the scan field.

For 18-bit position data, a value of 0 (all bits cleared) normally represents the far-left and far-bottom of the scan field; a value of 131072 represents the middle of the scan field; and a value of 262143 (all bits set) normally represents the far-right and far-top of the scan field.

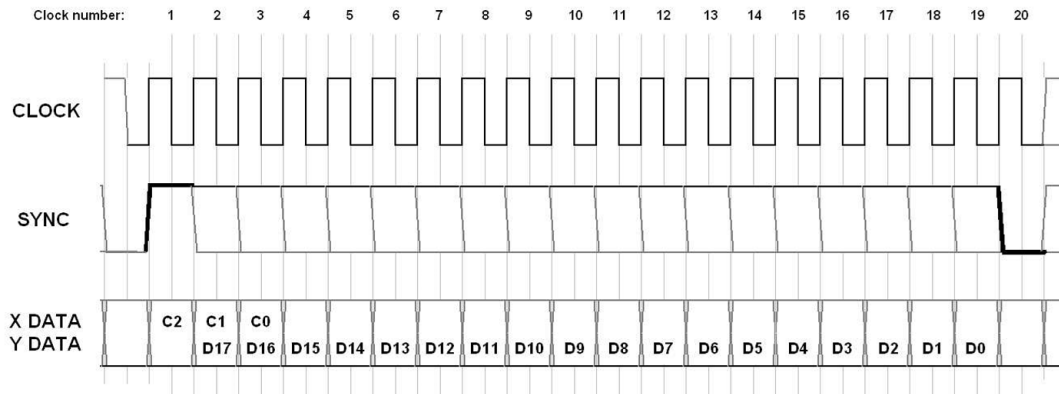


Figure 2: XY2-100 timing

6.2 FB4 Protocol

The FB4 protocol combines both X and Y into a single 32-bit data word, with X being the most significant 16-bits, and Y being the least significant 16-bits as shown below.

As is the case with XY2-100, signals that are sent to the Mach-DSP must change on the low-to-high transition of the clock; and signals are observed by the Mach-DSP on the high-to-low transitions of the clock. Below is an oscilloscope screen shot showing the simplest implementation of the FB4 protocol. Yellow is the clock. Pink is data. Blue is Frame Sync. The numeric data representation is the same as XY2-100. In the picture below, the most significant bits for both X and Y are high, and the rest of the bits are low. This represents +32768, which is equivalent to the center of the scan field.

Once the Frame Sync has been detected low, it can remain low during the next 31 clocks, or it may be brought high again after the first clock as shown above. In any event, Frame Sync must be brought high by clock number 32, as shown in the diagram above, otherwise a Frame Sync Error will occur, and data will not be communicated correctly. In essence, all of this means that the data is "framed" in 32-clock sequences.

The Clock line can be any frequency up to a theoretical maximum of 25MHz, with any reasonable duty cycle. The data frames described above may occur in sequence, one after another (called “coherent frames”), or they may be sporadic (called “isolated frames”). If they are sporadic, the Clock line may continue clocking in between data frames, or it may be quiescent (preferably held in a low state). In all cases, whether the clock is operational or not, the Frame Sync line must be held high when data is not sent.

The “frame rate” may be anything that the Clock and Frame Sync signals allow – ranging from slower than one frame per hour up to a theoretical maximum of 781,250 frames per second. The data communicated via this serial port is absorbed by the Mach-DSP at the rate that it is sent, and – unless Synchronous Sampling is selected (as described below), there is no attempt to synchronize the Mach-DSP’s servo sample clock (typically 100kHz to 300kHz) to the serial port signals.

You will note that 16-bits of data per axis are communicated using this protocol, and the entire sequence requires 32 clocks. X-axis data is first with the MSB during clock number 1. Y-axis data is next with MSB during clock number 17.

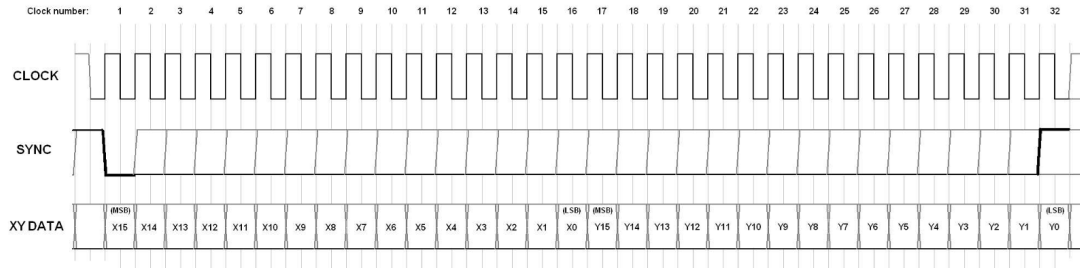


Figure 3: FB4 timing

6.3 Data Format Comparison

Parameter	XY2-100 (18 bits)	FB4
Data Resolution	18 bits	16 bits
Midpoint Value	131072 (2^{17})	32768 (2^{15})
Frame Size	19 bits $\times$ 2 channels	32 bits (combined)
Data Lines	2 (X and Y separate)	1 (X and Y multiplexed)
Max Amplitude	131071	32767

Table 3: Protocol comparison

7 Operation Guide

7.1 Initial Setup

1. Select the desired protocol (XY2-200.ino or FB4.ino) and configure the pin definitions
2. Configure the desired waveforms, frequencies, and amplitudes
3. Upload the firmware to the Arduino
4. Connect the Arduino to your scanner system following the appropriate pinout table Figure 1. provides and example for using an Arduino Portenta C33
5. Power on the system
6. Launch the Mach-DSP Servo Control Application. Select the desire Input Sorce (XY2-100 or FB4) Figure 4.
7. Note: make sure your systems has proper tuning parameters otherwise the system may under perform or case erratic movements

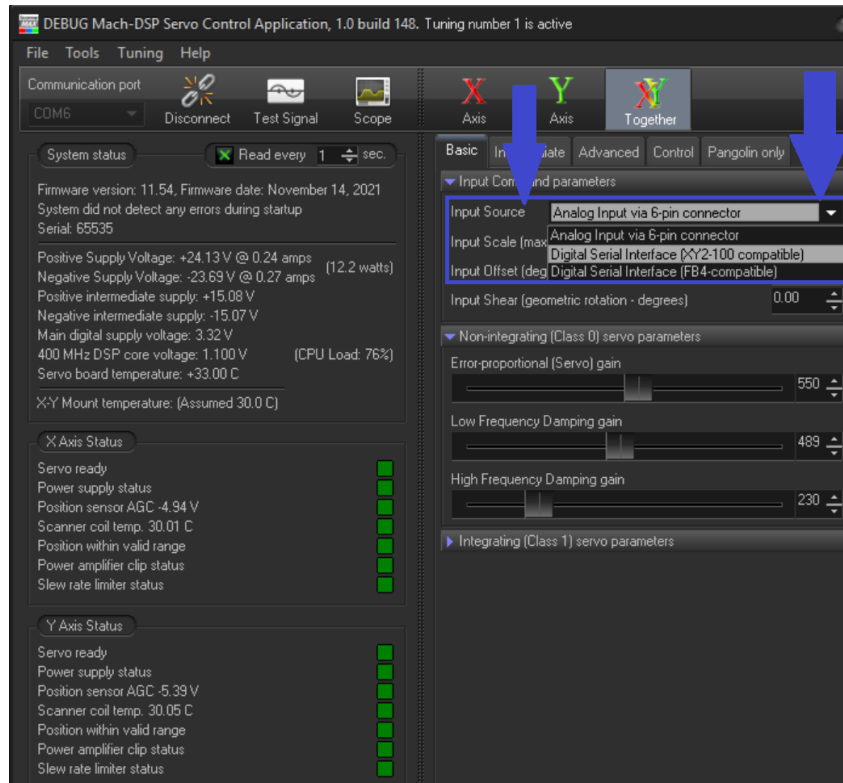


Figure 4: Input source

8 Appendices

8.1 Version History

- 1.0: Initial release
 - Basic XY2-100 protocol implementation
 - Multiple waveform support
 - Independent channel control
- 1.1: FB4 protocol support added
 - FB4 protocol implementation
 - 16-bit data format support
 - Single data line operation